📘 Día 11

Funciones

¿Qué es una función técnicamente?

En Python, una función es un **objeto de primera clase**. Esto significa que es un valor como cualquier otro (un número, un string):

1. Pueden ser retornadas por otras funciones: Al igual que una función puede devolver un número, puede devolver otra función. Esto es la base de los **clausuras (closures)** y los **decoradores**.

2. Pueden almacenarse en estructuras de datos: Puedes guardar funciones dentro de listas, diccionarios o tuplas. Esto es muy útil para crear "tablas de despacho" (dispatch tables) sin necesidad de usar muchos if/else.

3. Tienen atributos y métodos: Como son objetos reales (instancias de la clase function), las funciones tienen atributos propios que puedes consultar.

- __name__: El nombre de la función.
- __doc__: El docstring (comentario de documentación).
- Incluso puedes asignarles **atributos personalizados** (aunque no es una práctica común, es posible

4. Se pueden definir de forma anónima (lambda): Al ser valores, no siempre necesitan un "nombre" formal en el espacio de nombres. Puedes crear una función sobre la marcha y usarla donde la necesites.

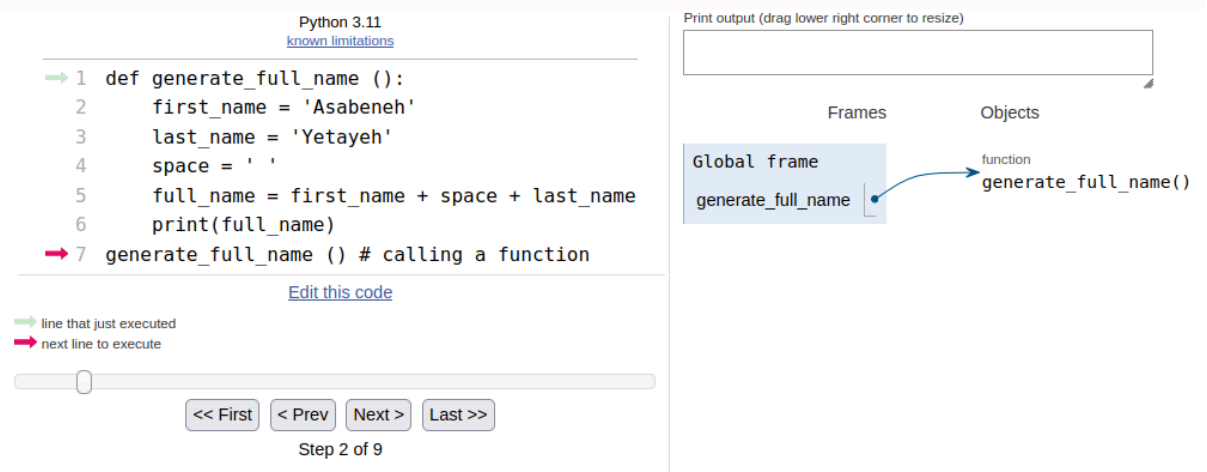
Cuando escribís `def`, Python crea ese objeto en memoria y lo asocia al nombre que le diste.

Función sin parámetros

```
def generate_full_name ():
    first_name = 'Asabeneh'
    last_name = 'Yetayeh'
    space = ' '
    full_name = first_name + space + last_name
    print(full_name)
generate_full_name () # calling a function
```

PASO 1 → Python lee la línea `def generate_full_name():`

En memoria (heap), Python crea un **objeto función**. En el *Global frame* (el espacio de nombres global), registra la variable `generate_full_name` apuntando a ese objeto.



🔍 **Lo que PyTutor omite:** El objeto función no es solo un puntero al código. Internamente contiene: el bytecode compilado (`__code__`), el nombre (`__name__`), las variables libres que captura del entorno (`__closure__`), y una referencia al entorno global (`__globals__`). En este caso `__closure__` es `None` porque no captura nada del exterior.

El cuerpo de la función **NO se ejecuta todavía**. Python solo lo compila a bytecode y lo guarda.

PASO 2 → Python llega a la línea `generate_full_name()`.

Cuando el intérprete de Python llega a la línea 7, no solo "salta", sino que realiza una serie de operaciones de gestión de memoria y flujo:

Búsqueda mediante LEGB: El intérprete busca el identificador `generate_full_name`. Siguiendo la regla **LEGB** (Local, Enclosing, Global, Built-in), En este contexto de nivel superior, el Global Frame actúa como el ámbito local actual.

L (Local):

Python pregunta: *¿Existe `generate_full_name` en el ámbito local actual?*

- **Respuesta:** Sí. Como estamos en el cuerpo principal del script, el ámbito local es el **Global Frame**.
- **Resultado:** Lo encuentra ahí porque fue definido en la línea 1.

E (Enclosing):

- Este paso se salta. Solo ocurre si estuviéramos dentro de una función que está dentro de otra función (funciones anidadas).

G (Global):

- Normalmente, si no estuviera en el Local, Python miraría aquí. Pero en la línea 7, **Local ya era el Global**, así que es como si ya hubiera revisado este cajón.

B (Built-in):

- Si no lo hubiera encontrado en el Global, Python buscaría en sus nombres reservados (como `print` o `len`).

Lo que trato de explicarte con esa frase es la **identidad** de los espacios de memoria en Python. Vamos a bajarlo a tierra con una analogía muy simple.

El concepto de "Módulos" (__main__)

Cuando tú abres un archivo .py y lo ejecutas, Python no lo ve simplemente como "líneas de texto". Lo envuelve en algo llamado **Módulo**. El archivo principal que ejecutas siempre recibe el nombre interno de **__main__**. Es como el "tablero principal" de tu programa.

¿Por qué locals() is globals() es True?

En Python, existen dos funciones integradas que te muestran qué variables conoce el intérprete en ese preciso instante:

- **globals()**: Es un diccionario que guarda todo lo que está en el "espacio global" (fuera de funciones).
- **locals()**: Es un diccionario que guarda lo que está en el "espacio donde estás parado ahora".

Aquí está el truco: Cuando estás en la línea 7 (fuera de cualquier función), **no hay nada más "local" que el propio archivo global.** No estás dentro de una caja (función), estás en el espacio abierto.

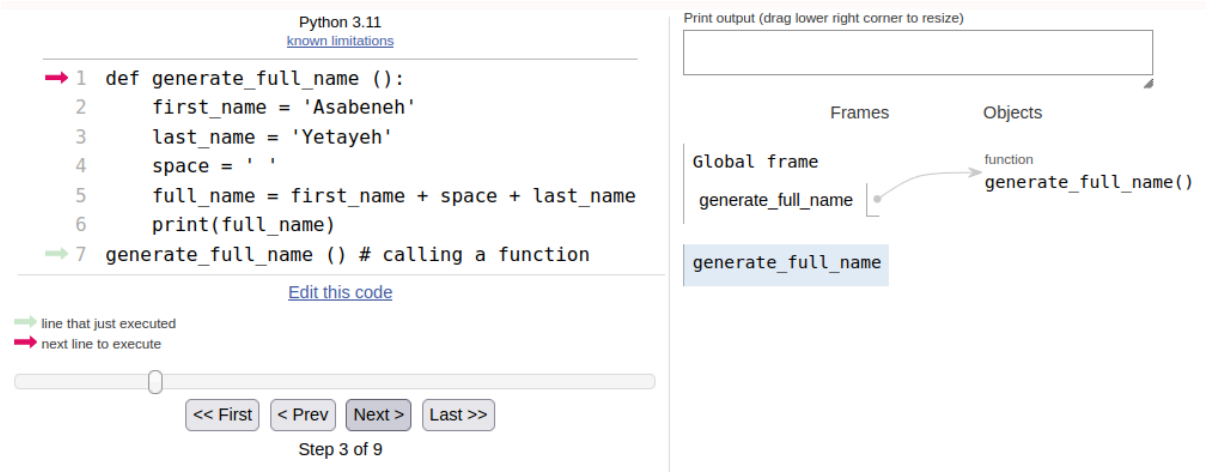
Por eso, para ahorrar memoria y lógica, Python dice:

> *"Si no estamos dentro de una función, no voy a crear una lista de variables 'locales' separada. Voy a usar la misma lista de variables 'globales' para ambas cosas".*

Por eso locals() is globals() devuelve **True**: porque **son el mismo objeto en la memoria**. Es la misma lista con dos nombres distintos.

Esto es una **llamada a función** (function call). Python:

1. Evalúa `generate_full_name` → encuentra el objeto función en el Global frame
2. Crea un nuevo **stack frame** (marco de pila) para esta ejecución
3. Suspende el contexto actual (el módulo principal) y salta al cuerpo de la función.



🔍 **Stack frame:** Es un bloque de memoria que Python apila en la **call stack** (pila de llamadas). Contiene: las variables locales de esa ejecución, una referencia al frame anterior (para saber a dónde volver), y el puntero de instrucción actual. En CPython esto es una estructura `PyFrameObject`.

PASO 3 → Se ejecuta `first_name = 'Asabeneh'`

Python crea el string `'Asabeneh'` en el heap (área de objetos). En el frame local de `generate_full_name`, registra la variable `first_name` apuntando a ese string.

🔍 **Los strings en Python son inmutables e internados parcialmente.** Strings cortos o que parecen identificadores pueden ser "internados" (guardados en un caché), pero en general cada literal crea un nuevo objeto string en el heap.

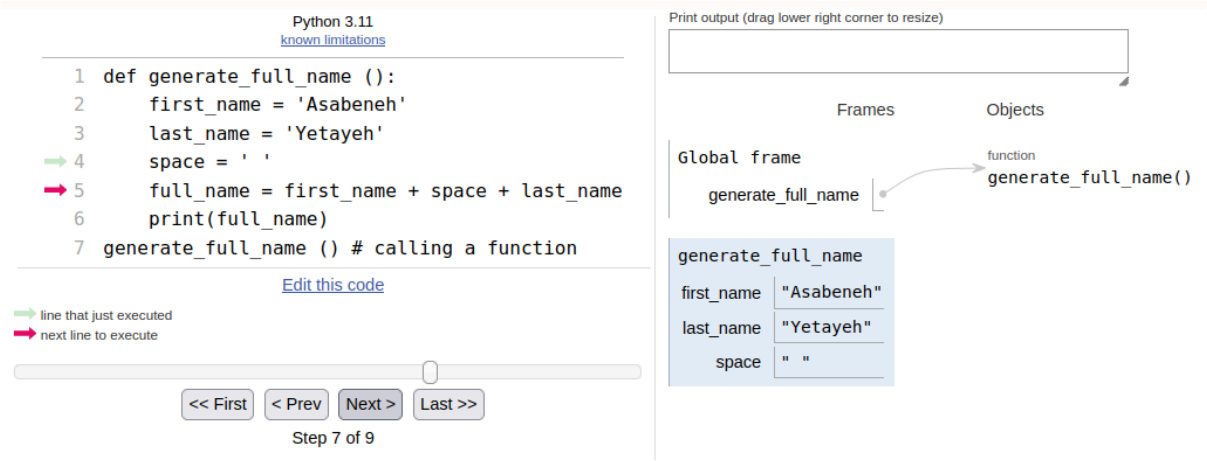
PASO 4 → Se ejecuta `last_name = 'Yetayeh'`

Igual que antes. Se crea otro objeto string `'Yetayeh'` en el heap y `last_name` apunta a él.

PASO 5 → Se ejecuta `space = ' '`

Se crea el string `' '` (un espacio). `space` apunta a él.

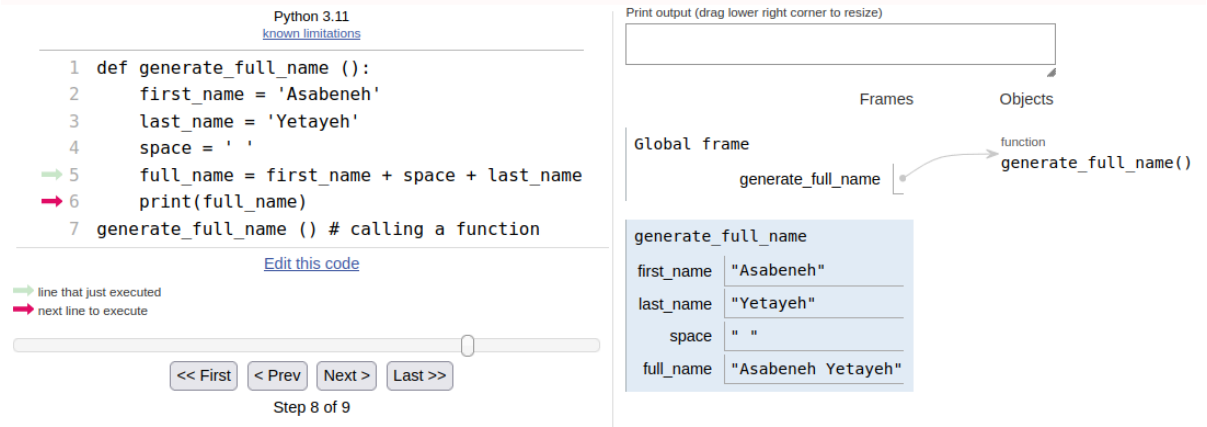
```
generate_full_name
  first_name ──▶ "Asabeneh"
  last_name  ──▶ "Yetayeh"
  space      ──▶ " "
```



PASO 6 → Se ejecuta `full_name = first_name + space + last_name`

Aquí ocurre **concatenación de strings**. Python evalúa de izquierda a derecha:

1. `first_name + space` → crea un **nuevo** objeto string `"Asabeneh "` en el heap
2. `"Asabeneh " + last_name` → crea **otro nuevo** objeto string `"Asabeneh Yetayeh"`
3. `full_name` apunta a ese resultado final

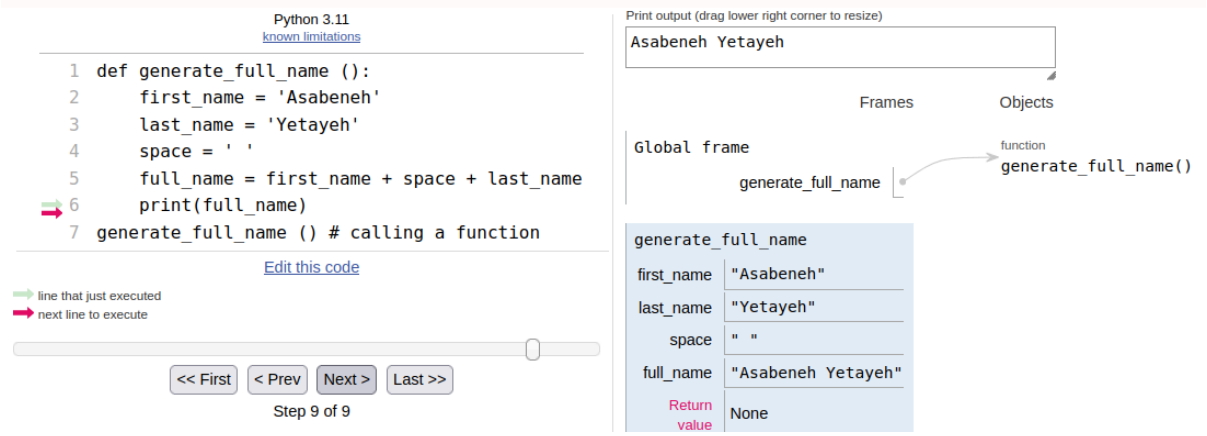


🔍 **Los strings intermedios son temporales.** El string `"Asabeneh "` creado en el paso 1 de la concatenación no tiene ninguna variable apuntándolo, entonces queda disponible para el **Garbage Collector (GC)** de Python. CPython usa **conteo de referencias**: cuando el contador llega a 0, el objeto se libera inmediatamente.

PASO 7 → Se ejecuta `print(full_name)`

`print` es una función built-in. Python la busca con LEGB primero en el frame local (no está), luego en el frame global (no está), luego en los **builtins** (`__builtins__`). La encuentra y la llama con `"Asabeneh Yetayeh"` como argumento. Imprime en `stdout`.

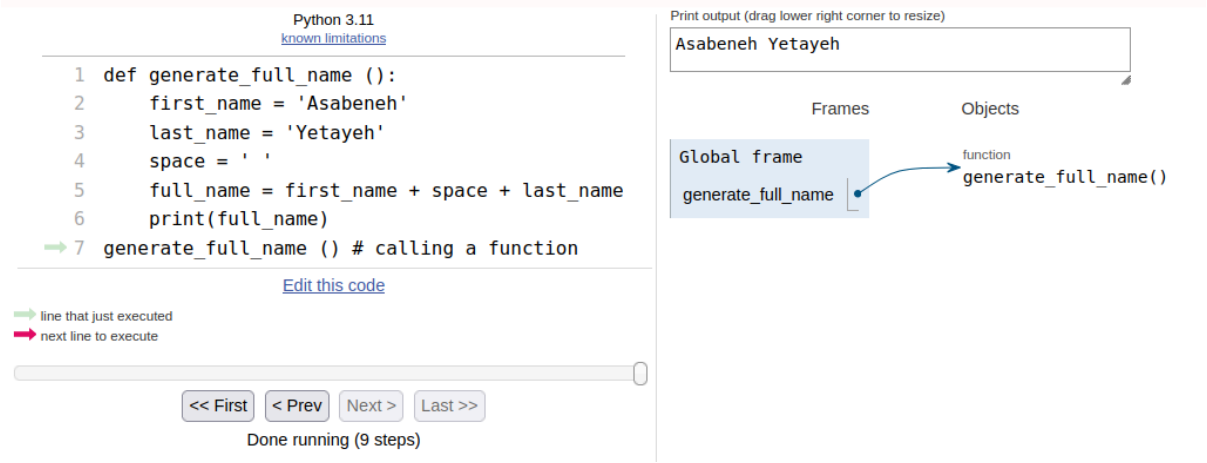
PASO 8 → La función termina (sin `return`)



Cuando una función no tiene `return` explícito, Python automáticamente retorna `None`. Esto es lo que PyTutor muestra como `Return value: None`.

🔍 **`None` en Python** es un singleton: existe un único objeto `None` en toda la ejecución. No se crea uno nuevo cada vez.

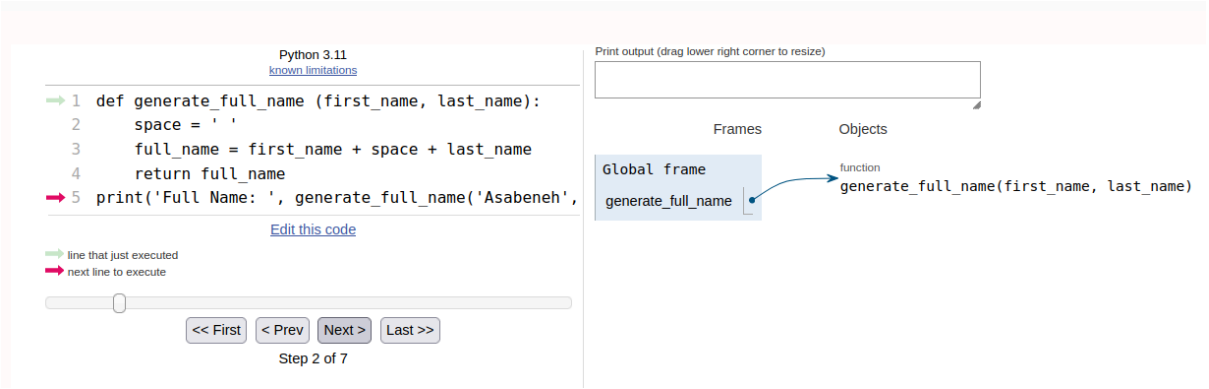
El frame local de `generate_full_name` es **destruido**. Todas las variables locales (`first_name`, `last_name`, `space`, `full_name`) dejan de existir. Los objetos string a los que apuntaban pierden esas referencias, y si nadie más los referencia, son recolectados por el GC.



Función como dos parámetro.

```
def generate_full_name (first_name, last_name):
   space = ' '
     full_name = first_name + space + last_name
     return full_name
print('Full Name: ', generate_full_name('Asabeneh','Yetayeh'))
```

Diferencias clave respecto al Ejemplo 1

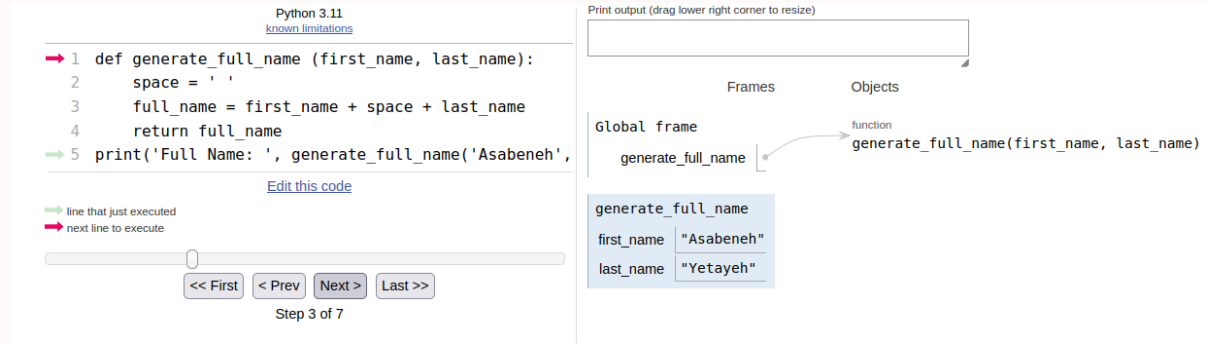


1. Parámetros vs Argumentos:

- **Parámetros** son los nombres en la definición: first_name, last_name
- **Argumentos** son los valores que pasás al llamar: 'Asabeneh', 'Yetayeh'

Cuando se ejecuta la llamada, Python crea el frame local y **vincula** cada parámetro con su argumento correspondiente por posición en el frame creado. Es exactamente como si al inicio del frame se ejecutara:

python
first_name = 'Asabeneh'
last_name = 'Yetayeh'



Paso por Valor o por Referencia? (El estilo Python)
En Python, no existe el "paso por valor" o "paso por referencia" tradicional (como en C++ o Java). Python utiliza "Pass by Object Reference" (paso por referencia de objeto) o "Pass by Assignment".

- ¿Cómo funciona? Cuando pasas 'Asabeneh' a la función, no estás pasando el texto en sí, ni una copia del texto. Estás pasando la dirección de memoria del objeto que vive en el Heap.
- En este caso (Strings): Como los strings son inmutables, se comportan *parecido* a un paso por valor (si intentas cambiarlos dentro, Python crea un objeto nuevo en lugar de modificar el original), pero técnicamente siempre estás pasando referencias a objetos.

Anatomía de la Memoria: Stack vs. Heap

Cuando ejecutas generate_full_name('Asabeneh', 'Yetayeh'), la memoria se organiza así:

A. El Heap (Memoria de Objetos)

Aquí es donde viven los datos reales. Antes de que la función empiece a trabajar, ya existen en el Heap los objetos:

- Objeto String: 'Asabeneh' (en la dirección, por ejemplo, 0x101)
- Objeto String: 'Yetayeh' (en la dirección 0x102)

B. El Stack Frame (Memoria de Ejecución)

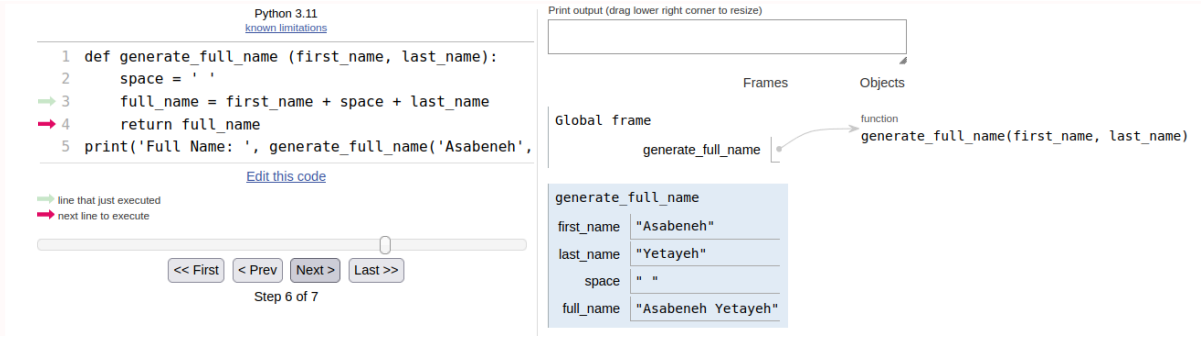
Al llamar a la función, se crea un nuevo **Stack Frame**. Los parámetros se almacenan **exclusivamente dentro de este frame** como variables locales.

- first_name: No contiene el texto, contiene el puntero 0x101.
- last_name: Contiene el puntero 0x102.

  ¿Los parámetros solo se almacenan en el frame? Sí y no. El **nombre** (el identificador first_name) y su **puntero** viven solo en el Stack Frame. Pero el **contenido** (los datos reales) vive en el Heap. Cuando la función termina y el frame se destruye, las etiquetas first_name y last_name desaparecen, pero los objetos en el Heap podrían seguir ahí (hasta que el *Garbage Collector* los limpie).

El proceso paso a paso en memoria

1. **Evaluación de argumentos:** Python ve 'Asabeneh' y 'Yetayeh', los busca (o crea) en el **Heap**.
2. **Creación del Frame:** Se crea el bloque de memoria en el **Call Stack** para generate_full_name.
3. **Vinculación (Binding):** Python asigna las direcciones de memoria del Heap a los nombres locales del Frame.
   - *Internamente:* Frame.local_vars['first_name'] = 0x101
4. **Ejecución:** La función usa esas referencias para realizar la concatenación.
5. **Retorno:** Se evalúa full_name, se envía la referencia del nuevo objeto creado al llamador (print) y el **Stack Frame se destruye inmediatamente**, liberando los nombres first_name y last_name.



2. return:

El proceso del return y la ejecución de print

Cuando el intérprete llega a la línea del return, se desencadena una coreografía de punteros y limpieza de memoria:

1. La sentencia return full_name

Dentro del **Stack Frame** de la función, la variable local full_name contiene una **dirección de memoria** (referencia) que apunta al objeto string "Asabeneh Yetayeh" situado en el **Heap**.

- Al ejecutar return, Python no envía el texto, envía esa **dirección de memoria** de vuelta al punto donde se llamó la función (la línea del print).
- Inmediatamente después de entregar la dirección, el **Stack Frame** de la función se marca para su destrucción y se elimina del Call Stack. Sin embargo, el objeto en el Heap sobrevive porque ahora el print lo está "sujetando".

2. Evaluación de la línea print(...) (Orden de ejecución)

Aunque nosotros leemos de izquierda a derecha, Python evalúa los argumentos de una función de **adentro hacia afuera**:

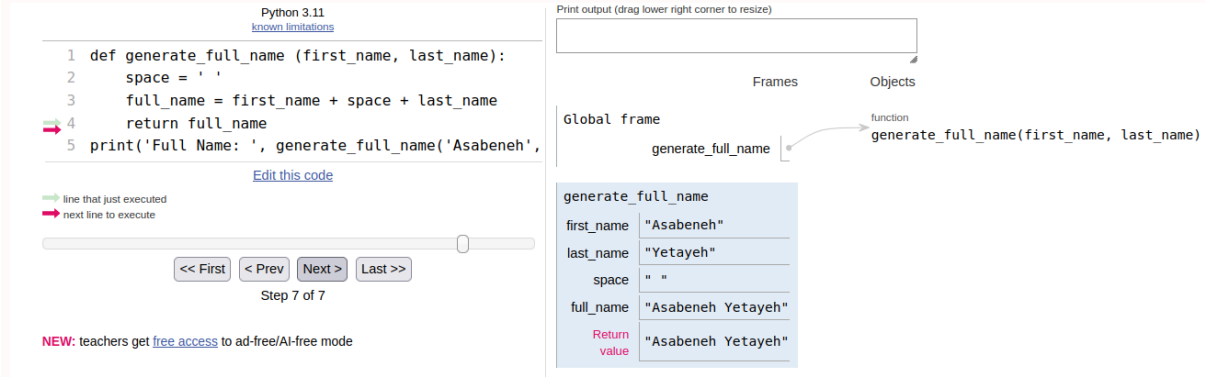
1. **Búsqueda de print (LEGB):** El intérprete busca el identificador print. No está en el Local (del módulo), ni en Enclosing, ni en Global. Lo encuentra finalmente en el ámbito **Built-in**.
2. **Preparación de argumentos:** Para que print pueda ejecutarse, Python primero debe resolver qué hay dentro de sus paréntesis:
    - **Argumento 1:** El literal 'Full Name: '. Python obtiene su dirección de memoria en el Heap.
    - **Argumento 2:** La llamada generate_full_name(...). Aquí es donde Python "salta" a la función, esta se ejecuta y, como vimos en el paso anterior, **retorna una dirección de memoria**.
3. **La llamada final:** Ahora que Python tiene las dos direcciones de memoria, la línea se ve internamente algo así: print(0xDireccion_Texto_Fijo, 0xDireccion_Resultado_Funcion)

3. Ejecución de print

La función print recibe estas referencias, accede a los valores reales en el **Heap**, los convierte a texto si es necesario y los envía a la consola separados por un espacio (su comportamiento por defecto).

Puntos clave para tu resumen técnico:

- **Paso de Referencias:** Tanto el return como el pasaje de argumentos a print funcionan mediante **direcciones de memoria**. Nunca se "mueve" el contenido real del objeto, solo se pasan los punteros.
- **Ciclo de Vida:** El objeto creado dentro de la función (full_name) no muere al terminar la función porque su dirección fue "rescatada" por el return y entregada al print.
- **LEGB para Built-ins:** Es vital notar que print es una función más, y Python la localiza en el último paso del algoritmo LEGB (**Built-in**).



EJEMPLO 3: Función como argumento (Higher-Order Function)

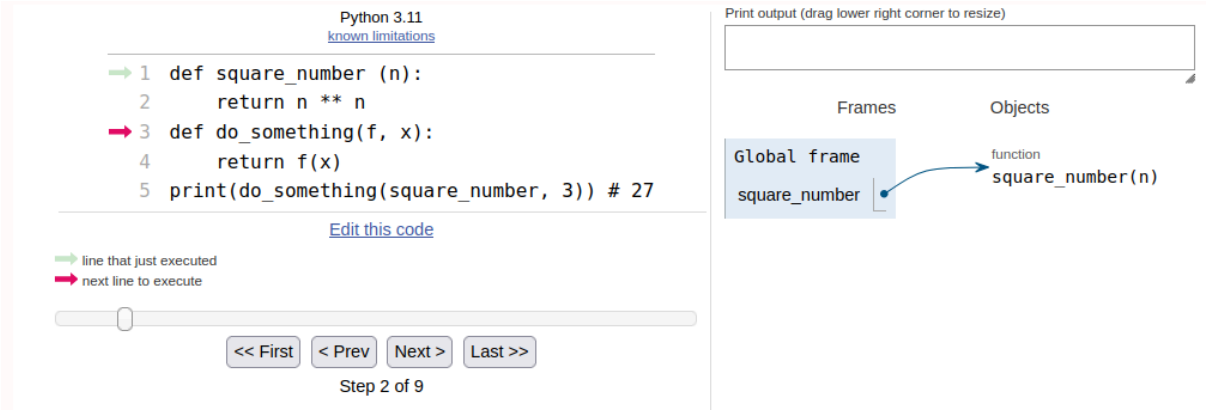
```
def square_number (n):
    return n ** n

def do_something(f, x):
    return f(x)

print(do_something(square_number, 3)) # 27
```

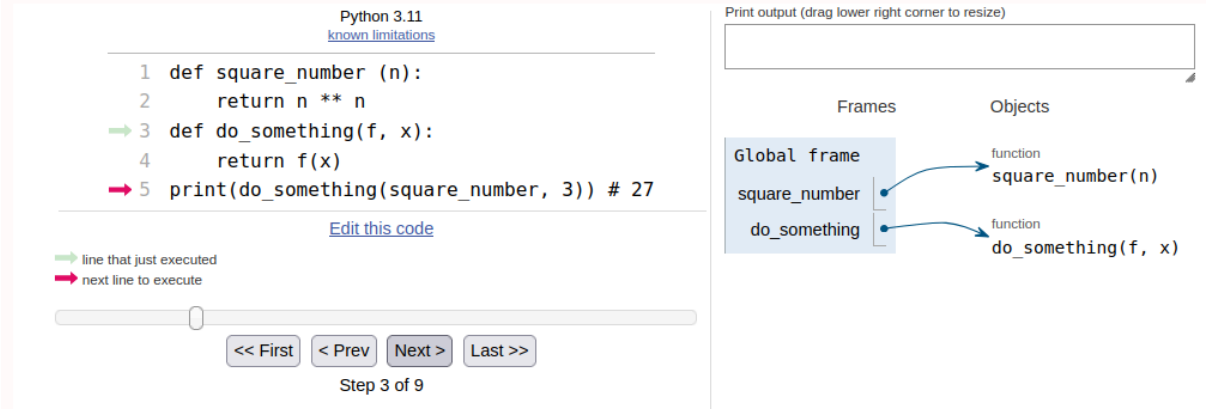
PASO 1 → Se definen ambas funciones

En el Global frame quedan dos variables:

Global frame

square_number ⟶ function square_number(n)
do_something ⟶ function do_something(f, x)

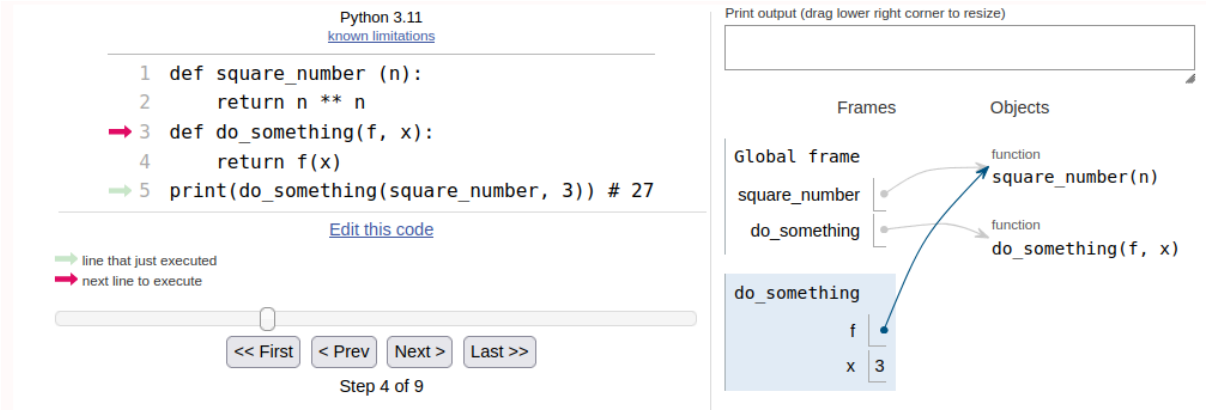




PASO 2 → Se llama do_something(square_number, 3)

Python pasa **el objeto función** square_number como valor. Recuerda: las funciones son objetos de primer orden. Se crea el frame de do_something:

do_something
f ⟶ function square_number(n) ← apunta al mismo objeto que en Global frame
x ⟶ 3

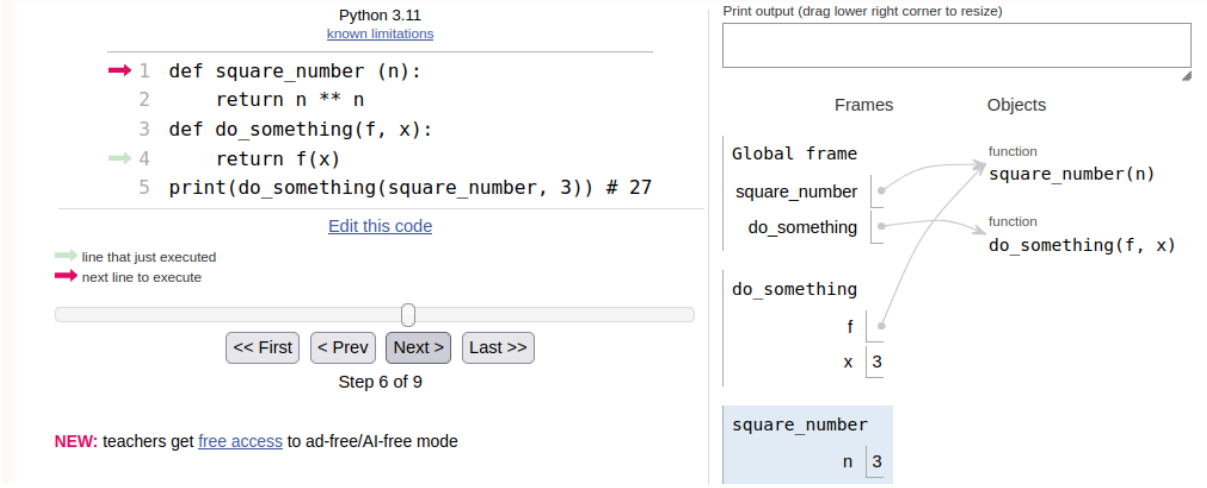


🔍 **Lo que PyTutor ilustra con la flecha:** La variable f dentro de do_something apunta **al mismo objeto función** que square_number en el Global frame. No es una copia. Son dos referencias al mismo objeto en el heap.

PASO 3 → Se ejecuta return f(x)

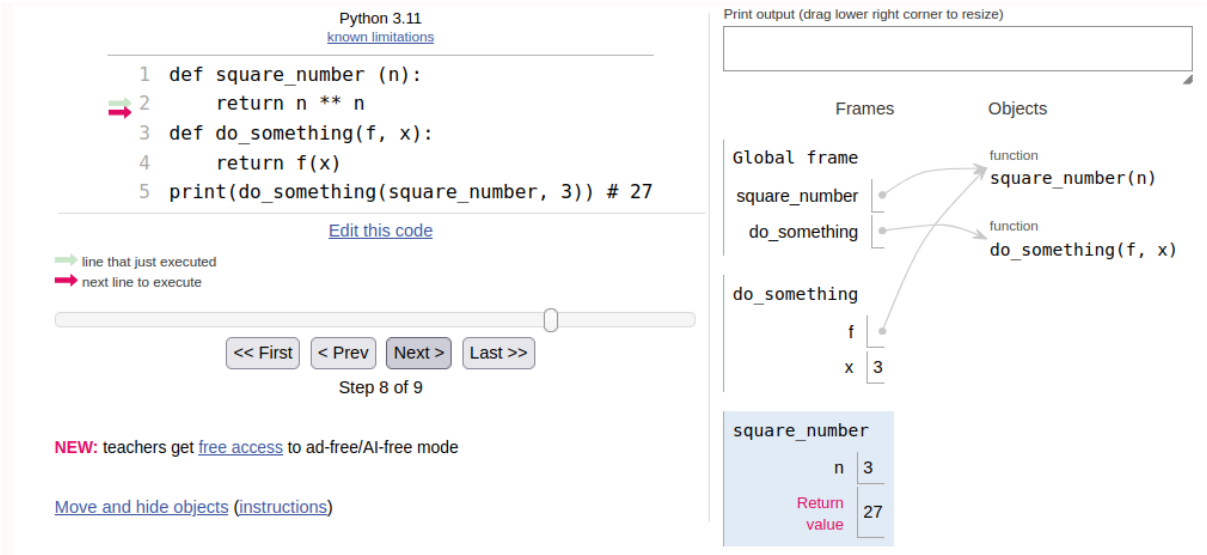
f es el objeto función square_number. f(x) es equivalente a square_number(3). Python crea un **nuevo frame** para square_number:

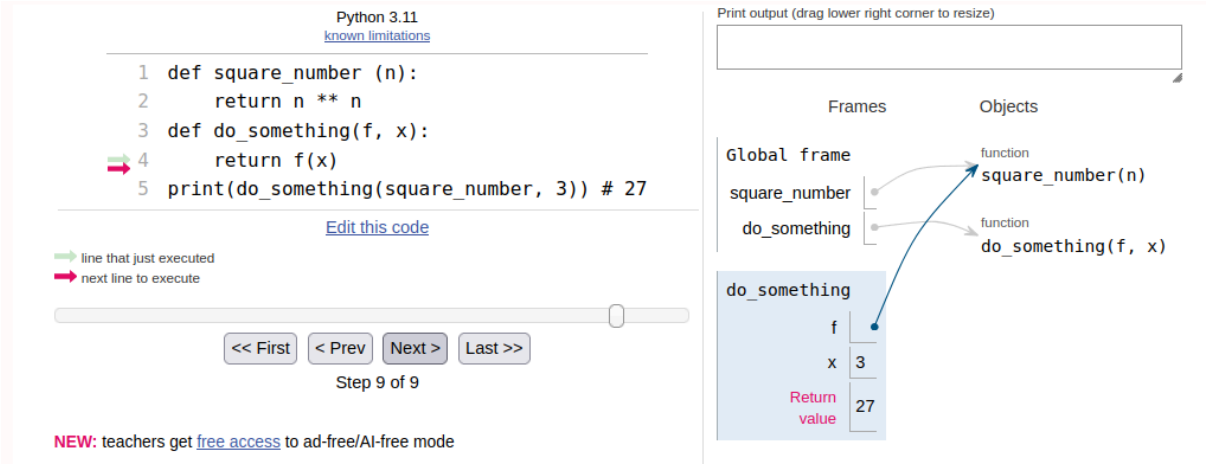
square_number
n ──▶ 3



PASO 4 → Se ejecuta return n ** n

3 ** 3 = 27. Python evalúa el operador de potencia. Se crea el objeto entero 27 en el heap. Se retorna.



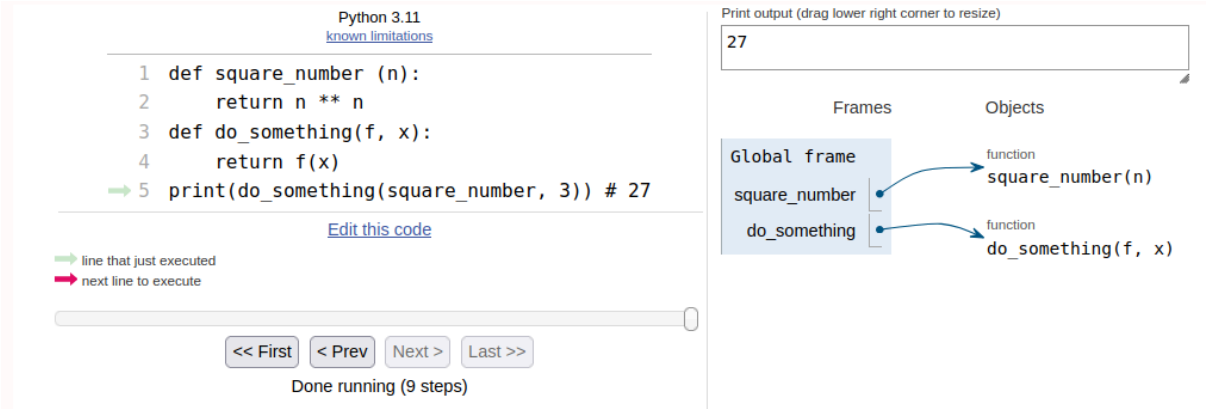


⚠️ **Nota técnica importante:** El código dice n ** n, no n ** 2. Entonces con n=3 el resultado es $3^3 = 27$, no $3^2 = 9$. El nombre square_number es engañoso, en realidad calcula n elevado a sí mismo.

Stack de llamadas en este momento (de abajo hacia arriba):

square_number ← tope (ejecutando ahora)
do_something
<módulo> ← base

Cuando square_number retorna 27, su frame se destruye. do_something recibe 27 y lo retorna. Su frame también se destruye. print recibe 27 e imprime.



EJEMPLO 4: `*args` - Número arbitrario de argumentos

Si no conocemos el número de argumentos que pasamos a nuestra función, podemos crear una función que puede tomar un número arbitrario de argumentos añadiendo * antes del nombre del parámetro.

Ejemplo:

```
def sum_all_nums(*nums):
    total = 0
    for num in nums:
        total += num     # same as total = total + num
  return total

print(sum_all_nums(2, 3, 5)) # 10
```

¿Qué hace el * técnicamente?

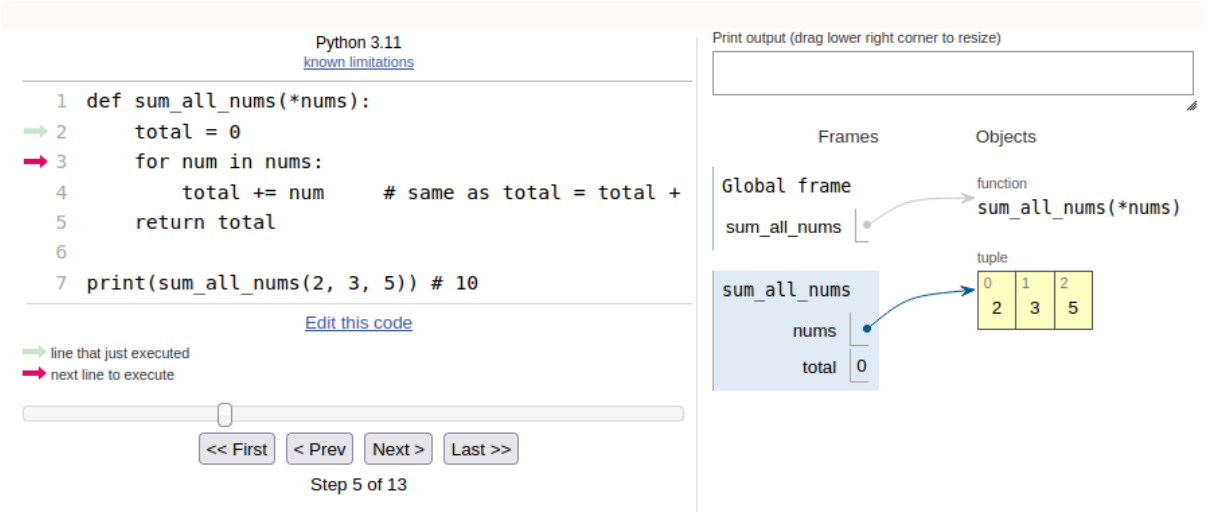
Cuando Python ve `*nums` en la definición, al recibir la llamada **empaqueta todos los argumentos posicionales en una tupla** y la asigna a `nums`.

python
sum_all_nums (2, 3, 5)
nums = (2, 3, 5) ← tupla, no lista

🔍 **Es una tupla, no una lista.** Las tuplas son inmutables. Podés iterar sobre `nums`, pero no podés hacer `nums.append(4)`. Esto es intencional: los argumentos que recibiste no deberían modificarse.

El frame en memoria:

sum_all_nums
  nums ──▶ (2, 3, 5) ← objeto tupla en el heap
  total ──▶ 0 ← al inicio



El bucle for num in nums:

1. ¿Cómo se recorre la tupla en memoria?

A diferencia de C o C++, donde un array de enteros guarda los valores contiguos (uno al lado del otro), en Python una **tupla** es un array de **punteros** (referencias).

Técnicamente, una tupla es una estructura llamada PyTupleObject. En su interior, tiene un array de punteros a PyObject.

El mecanismo de salto:

No se suma el "tamaño del dato" (porque el dato es un objeto complejo que vive en otro lado), sino el **tamaño de una referencia** (un puntero). En un sistema de 64 bits, cada puntero mide **8 bytes**.

1. **Dirección base:** La tupla sabe dónde empieza su array de punteros.
2. **Iteración 1:** El iterador toma la dirección base + offset 0. Lee el puntero que apunta al objeto 2. Asigna esa dirección a num.
3. **Iteración 2:** El iterador suma 8 bytes (el tamaño de un puntero) a la dirección anterior. Lee el puntero que apunta al objeto 3. Asigna esa dirección a num.
4. **Iteración 3:** Suma otros 8 bytes. Lee el puntero al objeto 5. Asigna esa dirección a num.

En resumen: Lo que "salta" de forma contigua en memoria son las direcciones de memoria (punteros), no los números en sí.

---

2. ¿Qué se le pasa a num en cada iteración?

En Python, **no se pasan valores, se pasan nombres (etiquetas)**. num es un nombre que "etiqueta" un objeto en el heap.

- **1ª Iteración:** num se convierte en un alias del objeto que vive en la primera posición de la tupla. Si el 2 vive en 0xABC, num ahora apunta a 0xABC.
- **2ª Iteración:** num se "despega" del 2 y se pega al objeto 3. El objeto 2 sigue vivo dentro de la tupla, pero num ya no lo conoce.
- **3ª Iteración:** num se mueve al objeto 5.

[NOTE] Al final del bucle, la variable num **no desaparece**. Se queda apuntando al último elemento (5) hasta que la función termine.

---

3. Alcance (Scope) y Ámbito (Namespace)

Estos términos suelen confundirse, pero en Python tienen matices específicos:

Ámbito (Namespace)

Es el "contenedor" técnico (un diccionario oculto) donde Python guarda los nombres de las variables y a qué objetos apuntan.

- En este caso, cuando llamás a sum_all_nums, se crea un **Ámbito Local**.

Alcance (Scope)

Es la región del código donde un nombre es visible y puede ser usado sin necesidad de prefijos (como self. o math.).

Variable	Alcance (Scope)	Ámbito (Namespace)
`total`	**Local** a la función. No existe fuera de ella.	Namespace de la función.
`num`	**Local** a la función.	Namespace de la función.
`nums`	**Local** (es el parámetro).	Namespace de la función.

Comportamiento de total y num:

1. **Definición:** total y num nacen en el momento en que se les asigna algo dentro de la función.
2. **Visibilidad:** Solo podés usarlas dentro del cuerpo de sum_all_nums. Si intentás hacer print(total) fuera de la función, Python lanzará un NameError.
3. **Vida útil (Lifetime):** Sus nombres viven en el stack mientras la función se está ejecutando. Una vez que la función llega al return total, el **Ámbito Local** se destruye.
    - El objeto final (el resultado 10) se devuelve al llamador.
    - Las etiquetas total, num y nums se borran.

- Si los objetos a los que apuntaban no tienen más referencias (como la tupla temporal creada por *args), el **Garbage Collector** los limpia de la memoria.

Un detalle ninja: A diferencia de otros lenguajes (como Java o C++), en Python los bucles for **no crean su propio alcance**. Por eso num sigue siendo accesible en cualquier parte de la función *después* de que el bucle terminó.

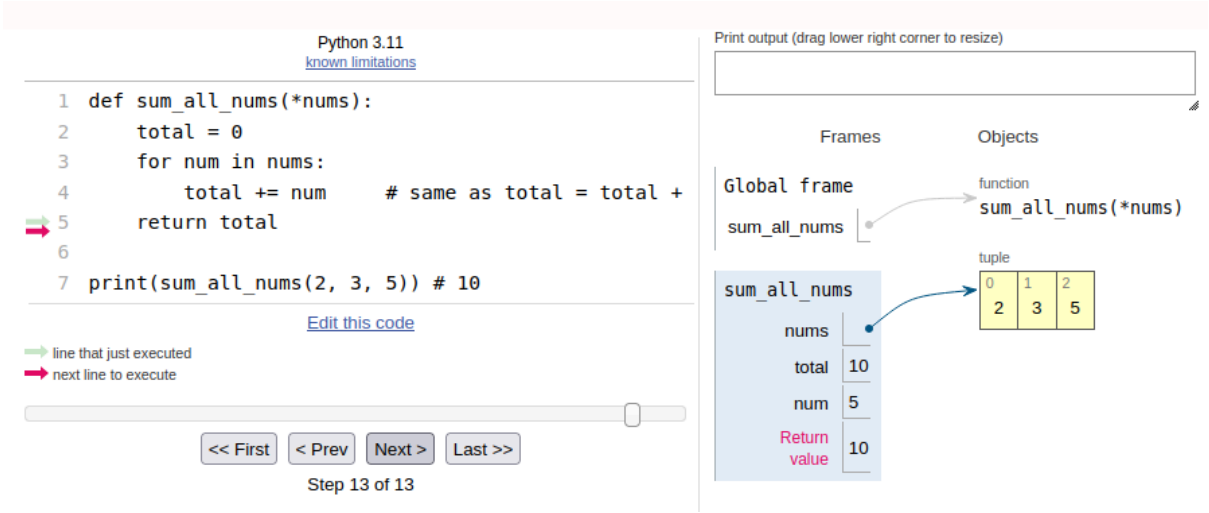
En cada iteración resumido:

- `num` apunta al elemento actual de la tupla
- `total += num` es azúcar sintáctico de `total = total + num`
  - Se crea un **nuevo objeto entero** con el resultado
  - `total` apunta al nuevo objeto
  - El objeto anterior queda sin referencias → GC lo recolecta

🔍 **Los enteros pequeños están cacheados.** CPython cachea los enteros del -5 al 256. Entonces `0`, `2`, `5`, `10` no se crean y destruyen realmente: ya existen en un pool permanente. Para números grandes sí se crean objetos nuevos.

Iteración a iteración:

total = 0
total = 0 + 2 = 2
total = 2 + 3 = 5
total = 5 + 5 = 10



Número predeterminado y arbitrario de parámetros en funciones

```
def generate_groups (team,*args):
    print(team)
    for i in args:
        print(i)
generate_groups('Team-1','Asabeneh','Brook','David','Eyob')
```

Aquí se mezclan un parámetro normal con `*args`:

- `team` recibe `'Team-1'` (primer argumento)
- `args` recibe `('Asabeneh', 'Brook', 'David', 'Eyob')` como tupla

Diccionario de desembalaje

Puede llamar a una función que ha nombrado argumentos usando un diccionario con nombres de clave coincidentes. Lo haces usando `**`.

```
# Define una función que recibe dos argumentos: 'nombre' y 'ubicación'
# Imprime un mensaje de saludo utilizando los argumentos proporcionados.
def greet(name, location):
    print("Hi there", name, "how is the weather in", location)

# Llamar a la función usando argumentos con nombre
# Salida: Hola Alice, ¿qué tiempo hace en Nueva York?
greet(name="Alice", location="New York")
```

1. El siguiente paso en memoria (Paso 3)

Cuando ejecutas greet(name="Alice", location="New York"), Python Tutor dibujará un nuevo bloque debajo del **Global frame**.

En el Stack (Frames):

Se crea el frame **greet**. Dentro de este frame, aparecen dos etiquetas (variables locales) inmediatamente:

- **name**: Se crea esta caja y se le asigna el puntero al objeto "Alice".
- **location**: Se crea esta caja y se le asigna el puntero al objeto "New York".

En el Heap (Objects):

- Aparecen dos nuevos objetos de tipo **String**: "Alice" y "New York".
- Las flechas salen del frame greet hacia estos objetos.

  [!IMPORTANT] En este punto, la ejecución se detiene en la línea 2 (print...), pero el frame ya está totalmente poblado.

---

2. Diferencia técnica: Keyword vs Positional

Aquí es donde se pone interesante. Aunque en el dibujo final de Python Tutor los dos se vean iguales, el "proceso de emparejamiento" que hace Python en la memoria cambia:

Característica	`greet("Alice", "New York")` (Posicional)	`greet(name="Alice", location="NY")` (Nombrado)
Lógica de asignación	**Por Orden:** Python dice: "El primero que recibí va a la primera variable de la definición".	**Por Etiqueta:** Python dice: "Buscá la variable que se llame `name` y ponele este valor".
Flexibilidad	Si cambias el orden, rompes el sentido (ej: saludas a New York en Alice).	Podrías escribir `greet(location="NY", name="Alice")` y funcionaría perfecto.
Carga en memoria	Es ligeramente más rápida. Python usa un array interno para asignar por índice (0, 1, 2...).	Requiere un paso extra de "mapeo". Python consulta un diccionario de nombres para saber qué valor va en qué caja.

3. ¿Qué cambia realmente en el "Unpacking"?

Si usaras el diccionario **my_dict que vimos antes, el proceso en memoria sería un híbrido:

1. **Paso A:** Python ve el **.
2. **Paso B:** Abre el diccionario y extrae las llaves como si fueran nombres de variables.
3. **Paso C:** Realiza la llamada como **Keyword Arguments** (Argumentos Nombrados).

En memoria (Python Tutor): No verías una diferencia visual en el frame de la función greet entre usar greet(name="Alice"...) o greet(**my_dict). El resultado final en el **Stack** es el mismo: una variable local llamada name apuntando a "Alice". La diferencia está en el **paso previo** de cómo Python obtuvo esos nombres.

Resumen visual del Paso 3:

Plaintext

```
  Frames                        Objects
+-------------------+           +-------------------------+
| Global frame      |           | function                |
|            greet  |---------->| greet(name, location)   |
+-------------------+           +-------------------------+
| greet             |           |                         |
|              name |---------->| "Alice"                 |
|          location |---------->| "New York"              |
+-------------------+           +-------------------------+
```

Al usar `name="Alice"`, le estás dando una instrucción explícita al intérprete: *"No te fijes en qué posición estoy, fijate en el nombre de la etiqueta"*.

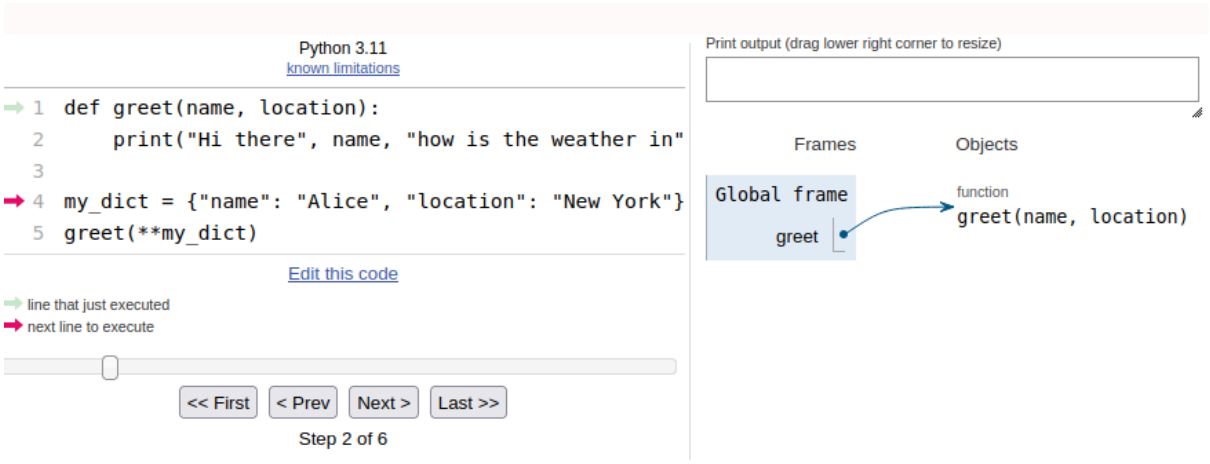
El operador ** al llamar

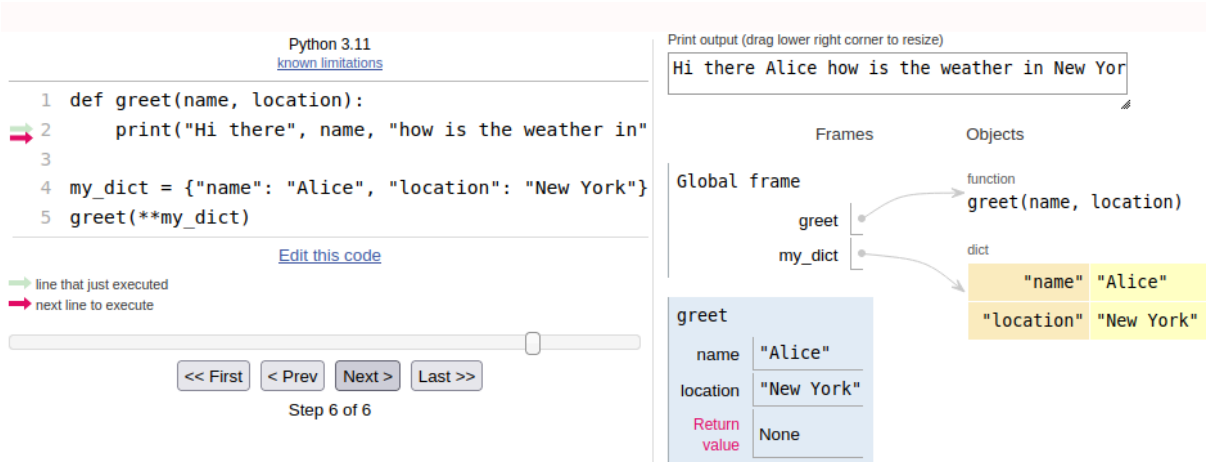
Define una función que recibe dos argumentos: 'nombre' y 'ubicación'
Imprime un mensaje de saludo utilizando los argumentos proporcionados.
```
def greet(name, location):
    print("Hi there", name, "how is the weather in", location)
```

Crea un diccionario con claves que coincidan con los nombres de los parámetros de la función.
```
my_dict = {"name": "Alice", "location": "New York"}
```

Llamar a la función usando el desempaquetado del diccionario
El operador ** desempaqueta el diccionario, pasando sus pares clave-valor como argumentos de palabra clave a la función.
```
greet(**my_dict)
```

my_dict **desempaqueta el diccionario como argumentos nombrados (keyword arguments). Es equivalente a escribir greet(**my_dict)





Esta es la parte donde muchos programadores se confunden, porque visualmente parece que el diccionario "vuela" hacia la función. Pero en Python, el operador ** no crea una tupla ni una lista; **es una instrucción de desempaquetado para el intérprete.**

1. La "Transformación" Técnica

Cuando escribís greet(**my_dict), **no se crea un objeto intermedio** (como una tupla o una lista). Lo que ocurre es un proceso de **mapeo de argumentos**:

1. **Iteración interna:** Python recorre el objeto diccionario my_dict que vive en el **Heap**.

2. **Extracción de pares:** Extrae la clave "name" y su puntero ("Alice"), y la clave "location" y su puntero ("New York").
3. **Inyección en el Stack:** Python coloca esos punteros directamente en los "slots" de memoria que la función greet ya tiene reservados para sus parámetros.

   Diferencia con *args: Mientras que el * (un solo asterisco) **sí** empaqueta todo en una **Tupla**, el ** (doble asterisco) en la llamada funciona como un **distribuidor**.

Representación Visual del paso al Frame.

```
       STACK (Frames)                        HEAP (Objetos)
+--------------------------+          +----------------------------------+
| GLOBAL FRAME             |          |  dict: my_dict                   |
|   my_dict  --------------|--------> |  { "name": id1, "location": id2 }|
+--------------------------+          +----------------------------------+
             |                                  |            |
      [ EL PUENTE ** ]                          |            |
             |                                  v            v
+--------------------------+               +-----------+  +--------------+
| FRAME: greet             |               | id1:      |  | id2:         |
|   name     --------------|-------------->| "Alice"   |  | "New York"   |
|   location --------------|-------------->| (String)  |  | (String)     |
+--------------------------+               +-----------+  +--------------+
```

En qué se convierte.

Técnicamente, greet(**my_dict) se convierte en una serie de **instrucciones de bytecode** llamadas CALL_FUNCTION_EX.

- **No se crea una tupla.** (Las tuplas son para argumentos posicionales *args).
- **No se crea un nuevo diccionario.** (A menos que la función esté definida como def greet(**kwargs), pero ese es otro tema).
- **Se convierte en una "Llamada por Palabra Clave" (Keyword Call):** El intérprete simplemente apila los valores en el orden en que la función los espera, usando los nombres como guía para no equivocarse de "cajón".

Número arbitrario de argumentos nombrados

También puede definir una función para aceptar un número arbitrario de argumentos con nombre.

```
def arbitrary_named_args(**args):
    print("Recibí un número arbitrario de argumentos, que suman un total de", len(args))
    print("Se proporcionan como un diccionario en mi función:", type(args))
    print("Vamos a imprimirlas:")
    for k, v in args.items():
        print(" * key:", k, "value:", v)
```

En general, evite esto a menos que se requiera, ya que hace que sea más difícil entender lo que la función acepta y hace.

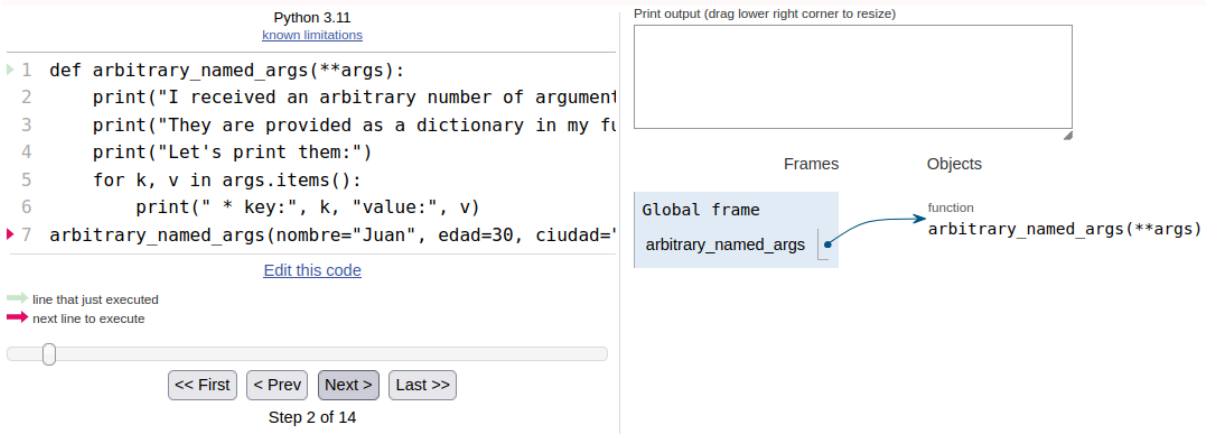
Python busca en el diccionario las claves que coincidan con los nombres de los parámetros. Si hay una clave que no tiene parámetro correspondiente → TypeError.

```
def arbitrary_named_args(**args):
    for k, v in args.items():
        print(" * key:", k, "value:", v)
```

**kwargs en la definición

Al igual que *args empaqueta posicionales en tupla, **kwargs empaqueta todos los keyword arguments en un **diccionario**.

```
arbitrary_named_args(nombre="Juan", edad=30, ciudad="Mendoza")
# args = {"nombre": "Juan", "edad": 30, "ciudad": "Mendoza"}
```



Python 3.11
known limitations

```
1 def arbitrary_named_args(**args):
2     print("I received an arbitrary number of argument
3     print("They are provided as a dictionary in my fu
4     print("Let's print them:")
5     for k, v in args.items():
6         print(" * key:", k, "value:", v)
7 arbitrary_named_args(nombre="Juan", edad=30, ciudad='
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 3 of 14

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
arbitrary_named_args → function: arbitrary_named_args(**args)

arbitrary_named_args
args → dict:

"nombre"	"Juan"
"edad"	30
"ciudad"	"Mendoza"

Python 3.11
known limitations

```
1 def arbitrary_named_args(**args):
2     print("I received an arbitrary number of argument
3     print("They are provided as a dictionary in my fu
4     print("Let's print them:")
5     for k, v in args.items():
6         print(" * key:", k, "value:", v)
7 arbitrary_named_args(nombre="Juan", edad=30, ciudad='
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 8 of 14

Print output (drag lower right corner to resize)

```
I received an arbitrary number of arguments,
They are provided as a dictionary in my func
Let's print them:
```

Frames | Objects

Global frame
arbitrary_named_args → function: arbitrary_named_args(**args)

arbitrary_named_args
args → dict:

"nombre"	"Juan"
"edad"	30
"ciudad"	"Mendoza"

k: "nombre"
v: "Juan"

Python 3.11
known limitations

```
1 def arbitrary_named_args(**args):
2     print("I received an arbitrary number of argument
3     print("They are provided as a dictionary in my fu
4     print("Let's print them:")
5     for k, v in args.items():
6         print(" * key:", k, "value:", v)
7 arbitrary_named_args(nombre="Juan", edad=30, ciudad='
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 9 of 14

Print output (drag lower right corner to resize)

```
I received an arbitrary number of arguments,
They are provided as a dictionary in my func
Let's print them:
 * key: nombre value: Juan
```

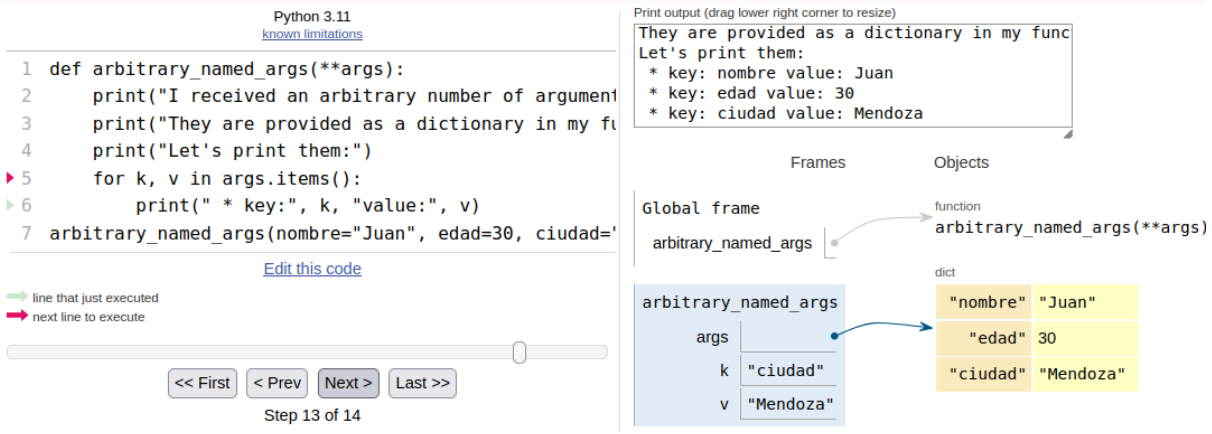
Frames | Objects

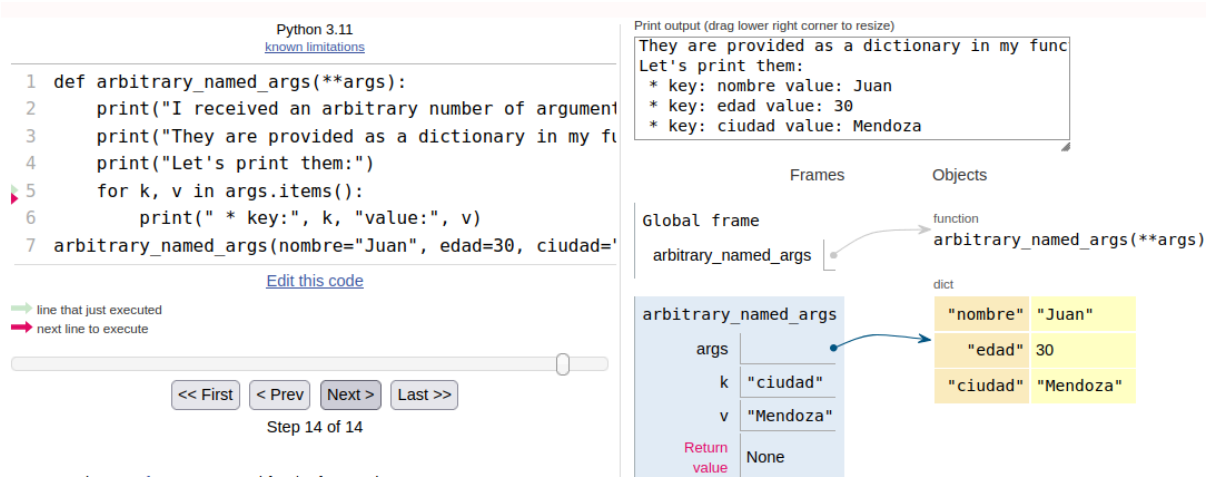
Global frame
arbitrary_named_args → function: arbitrary_named_args(**args)

arbitrary_named_args
args → dict:

"nombre"	"Juan"
"edad"	30
"ciudad"	"Mendoza"

k: "nombre"
v: "Juan"





Este es el escenario opuesto al anterior. Mientras que en el "desembalaje" (`**` en la llamada) desarmábamos un objeto, aquí el `**` en la **definición** funciona como un **imán o una red de pesca**.

Vamos a ver cómo se comporta la memoria paso a paso cuando ejecutas:
`arbitrary_named_args(nombre="Juan", edad=30, ciudad="Mendoza")`

---

1. El Momento de la Llamada (Paso "Cero")

En el **Global Frame**, Python evalúa los valores que estás pasando.

- Crea (o busca en el cache) el objeto `30`, el string `"Juan"` y el string `"Mendoza"` en el **Heap**.
- En este punto, esos valores son "argumentos sueltos" con una etiqueta temporal en la línea de comandos.

2. El Salto a la Función (La Magia del `**`)

Cuando el intérprete ve que la función está definida como `def arbitrary_named_args(**args):`, sucede lo siguiente:

1. **Creación de un Nuevo Objeto:** Python **instancia un nuevo objeto Diccionario** en el Heap. Este diccionario está vacío al principio.
2. **Empaquetado:** Python toma cada "Keyword Argument" de la llamada (`nombre=...`, `edad=...`) y los mete dentro de ese nuevo diccionario.
   - La clave es el nombre del argumento (un string `"nombre"`).
   - El valor es el puntero al objeto correspondiente (`"Juan"`).
3. **Asignación al Stack:** Se crea el **Frame local** de la función. Dentro, se crea la variable `args`, cuyo puntero apunta directamente al **nuevo diccionario** que acaba de crear.

Plaintext

```
    STACK (Frames)                      HEAP (Objects)
-------------------------+          +-----------------------------------+
GLOBAL FRAME             |          |  (Function: arbitrary_named_args)  |
  arbitrary_... ---------|--------> |                                   |
-------------------------+          +-----------------------------------+
           |                                     ^
           v                                     |
-------------------------+          +-----------------------------------+
FRAME: arbitrary_args    |          |  dict: (Este es el objeto 'args')  |
  args    ---------------|--------> |  {                                |
-------------------------+          |    "nombre": ref(id10), -----------|--+
                                     |    "edad":   ref(id20), -----------|--|--+
                                     |    "ciudad": ref(id30)  -----------|--|--|--+
                                     |  }                                |  |  |  |
                                     +-----------------------------------+  |  |  |
                                                                            |  |  |
                                     +-------------------------+            |  |  |
                                     | id10: "Juan" (Str)      | <----------+  |  |
                                     +-------------------------+               |  |
                                     | id20: 30 (Int)          | <-------------+  |
                                     +-------------------------+                  |
                                     | id30: "Mendoza" (Str)   | <----------------+
                                     +-------------------------+
```

4. Resolviendo tus dudas técnicas

- **¿Se crea otra estructura? SÍ**. A diferencia del primer ejemplo donde solo pasábamos punteros, aquí Python **crea un objeto Diccionario nuevo** en el Heap específicamente para recolectar esos argumentos.
- **¿Es una tupla? NO**. El `*args` (un solo asterisco) crea una tupla. El `**kwargs` (doble asterisco) siempre crea un **Diccionario**. Esto es porque los argumentos nombrados necesitan una clave (el nombre) para ser recuperados.

- **¿Qué pasa con las referencias?** * El diccionario `args` ahora es el "dueño" de las referencias a los objetos `"Juan"`, `30` y `"Mendoza"`.
    - El conteo de referencias de esos objetos aumenta en 1.
- **El alcance (Scope):** El objeto diccionario vive en el Heap, pero el nombre `args` solo existe en el **Scope Local** de la función. Cuando la función termina y el Frame desaparece, el diccionario se queda con 0 referencias y el **Garbage Collector** lo elimina (y con él, bajan los contadores de los objetos que contenía).

Diferencia clave que preguntaste:

En el ejemplo anterior (`greet(**my_dict)`), el diccionario **ya existía** y lo desarmábamos para llenar variables locales fijas. En este ejemplo (`**args` en la definición), las variables **vienen sueltas** y Python **las arma** en un diccionario nuevo porque la función no sabe cuántas vendrán.

> [!TIP] **Dato Pro:** Aunque el usuario le llame `args`, por convención en Python se suele usar `**kwargs` (keyword arguments) para no confundirlo con `*args` (positional arguments). Pero técnicamente, el nombre de la variable puede ser cualquiera, ¡lo que importa son los dos asteriscos!

Resumen visual del modelo de memoria de Python

```
STACK (frames)        HEAP (objetos)

_________________     ____________________________________

Global frame          function generate_full_name

  generate_full_name ──▶  __code__, __name__, ...

  square_number      ──▶ function square_number

  do_something       ──▶ function do_something


do_something frame     string "Asabeneh Yetayeh"

  f ──────────────────────────▶ (mismo objeto función de arriba)

  x ──────────────────────────▶ int 3


square_number frame
```

n ──────────────────────► int 3 (mismo objeto)

Reglas clave que siempre aplican:

1. Las variables son **referencias** (punteros), nunca contienen el objeto directamente
2. Asignar `a = b` hace que ambas apunten al mismo objeto, no copia
3. Los frames locales se crean al llamar y se destruyen al retornar
4. Sin `return`, la función siempre retorna `None`
5. `*args` → tupla, `**kwargs` → diccionario